Correspondence

Machine Learning Based Spectrum Allocation for THz UAV Communications

***Abstract*— Terahertz (THz) links between unmanned aerial vehicles (UAVs) offer very wide bandwidth, but molecular absorption splits the band into frequency-dependent transmission windows, and mobility misaligns the narrow beams. Which frequency channels to activate, and how to share the transmit power among them, therefore decides the achievable capacity. An existing threshold-search method, MaxActive, returns the best channel set within its search family but must repeat its search over thousands of channels at every geometry update. This correspondence investigates machine-learning alternatives that work from the same physical channel state. Eighteen supervised regressors from six model families predict the capacity that the search would deliver, and a dueling double deep Q-network, formulated as a single-step contextual bandit, learns the channel-selection decision itself. All methods are evaluated with data regenerated from a physics-consistent THz model under four mobility patterns, using leakage-safe nested cross-validation. A Gaussian process predicts the capacity most accurately, whereas single trees, plain boosting, and unregularized linear regression overshoot the physical bound. The learned policy attains at least 99.7% of the reference capacity at every evaluated geometry, but it is trained separately for each operating context, with the reference capacity in its reward. Once trained, both learned branches lower the online decision cost by about two orders of magnitude or more and occupy kilobytes of memory.**

***Index Terms*— Beam misalignment, channel selection, contextual bandit, deep reinforcement learning, Gaussian process, machine learning, spectrum allocation, terahertz communications, unmanned aerial vehicles.**

I. INTRODUCTION

TERAHERTZ (THz) frequencies (0.1–10 THz) offer contiguous bandwidths of tens to hundreds of gigahertz, which makes them a candidate band for sixth-generation (6G) wireless systems and, in particular, for high-capacity links between unmanned aerial vehicles (UAVs) such as drone swarms and aerial backhaul [1]–[3]. At the altitudes where drones operate, the water-vapor concentration is lower than at sea level, so THz links are more practical in the air than on the ground [4], [5]. THz propagation, however, differs from that at microwave and millimeter-wave frequencies in three ways that matter for system design. First, the free-space spreading loss grows with

Manuscript received XXXXX 00, 0000; revised XXXXX 00, 0000; accepted XXXXX 00, 0000.
This work was supported by the Scientific and Technological Research Council of Türkiye (TÜBİTAK) under Grant No. 224N014. *(Corresponding author: Korhan Erdoğdu.)*

Korhan Erdoğdu, Özgür Gürbüz, and Mikail Erdem are with the Faculty of Engineering and Natural Sciences, Sabancı University, Istanbul 34956, Türkiye (e-mail: korhan.erdogdu@sabanciuniv.edu; ozgur.gurbuz@sabanciuniv.edu; mikail@sabanciuniv.edu). Akhtar Saeed is with Kadir Has University, Istanbul, Türkiye (e-mail: akhtarsaeed@sabanciuniv.edu).

0018-9251 © 2026 IEEE

the square of both distance and frequency [6]. Second, water-vapor molecules absorb energy at many discrete resonance lines, so the attenuation changes by orders of magnitude across the band; the usable spectrum consists of transmission windows whose width shrinks as the distance grows [6], [7]. Third, the absorbed energy is re-radiated as *molecular absorption noise*, so the noise level also depends on frequency and distance [6]. The link budget is closed with highly directional antennas, but narrow beams are easily misaligned when the aircraft move [8]. As a result, the capacity of a UAV-to-UAV THz link depends jointly on distance, absorption, noise, antenna gain, beamwidth, and beam alignment, and it can collapse by many orders of magnitude when the beams drift apart [9].

A. Spectrum Allocation in the THz Band

These properties make *spectrum allocation* a central design problem. The band is divided into many narrow frequency channels, and for every channel geometry the transmitter must decide which channels to activate and how to divide its fixed transmit power among them. Activating a channel adds bandwidth, but it also takes power away from the others. A channel near an absorption line contributes little capacity and much noise, whereas concentrating the power on too few channels wastes bandwidth. The best choice changes whenever the distance or the beam alignment changes, so in a mobile UAV link the decision has to be recomputed continuously.

B. Related Work

Optimization-based allocation. Han *et al.* [7] exploited the distance-dependent transmission windows with a multi-wideband waveform and optimized the sub-band allocation to extend the communication range under rate and power constraints. Shafie *et al.* [10] designed adaptive sub-band bandwidths for multiuser THz systems, which reduces the spread of absorption loss across users. Both target terrestrial links and do not model aerial mobility. For aerial platforms, Saeed *et al.* [5] proposed the common flat band (CFB) scheme, which keeps the channels where path gain and noise spectral density are both flat, and analyzed the capacity of THz links for drones, aircraft, and satellites. Xu *et al.* [11] jointly optimized UAV location, bandwidth, and power, but for UAV-to-ground links. Beam-oriented works, such as LeTera [12] and LeBeam [8], adapt the beamwidth to in-flight mobility, but they assume a single THz channel and therefore do not address channel selection. Saeed *et al.* [9] combined these strands for drone-to-drone links: they jointly considered channel selection, water-filling (WF) or equal-power (EP) allocation, and beamwidth adjustment, and proposed the *MaxActive* channel-selection scheme. MaxActive with EP achieved a capacity close to that of WF-based schemes while outperforming CFB and standard full-band selection, and it is the reference solution adopted here. Its main practical drawback is computational. The search re-evaluates the capacity over the whole band for many candidate channel sets, and the joint allocation loop must be repeated after every geometry change.

Learning-based allocation. Machine learning (ML) has been used to replace costly resource-allocation algorithms with fast learned mappings. Sun *et al.* [13] showed that a neural network trained to imitate an iterative power-control algorithm can reproduce its output at a small fraction of its run time, the "learning to optimize" paradigm. Deep reinforcement learning (DRL) has been applied widely to wireless resource management [14]; for example, multi-agent deep Q-learning has been used for dynamic power allocation [15]. For THz systems, Shafie *et al.* [16] trained a deep network without labels to approximate a sub-band allocation; as noted in [17], such schemes are sensitive to training and can occasionally violate hard power or bandwidth constraints. Hu and Han [18] recently combined graph neural networks with DRL to allocate power and antenna sub-arrays in dynamic THz UAV networks; their objective is resource efficiency rather than per-link channel selection. Surveys of ML for THz communications are given in [19]. Closest to the present work, Elçik *et al.* [17] formulated THz drone-to-drone channel selection as a contextual bandit solved by a dueling double deep Q-network (D3QN). Their policy matched the MaxActive capacity with a much lower operation count, but it requires a separately trained model for every distance–gain pair, and it was tested on a narrower band (0.5–1 THz) with a single learning method.

C. Research Gap and Contributions

Two questions remain open. First, is a deep-RL policy necessary, or can simpler supervised models, trained on the same physical channel state, predict what the spectrum-allocation search achieves? Second, what do these alternatives cost in accuracy, physical feasibility, run time, memory, and training effort, under a validation protocol that is credible for the small datasets a physics simulator produces? This correspondence addresses both questions for the MaxActive spectrum allocation problem of [9]. The contributions are as follows.

1) A physics-consistent reproduction of the MaxActive capacity under WF and EP allocation for four UAV mobility patterns, including mobility over elevation and measured windy-hover acceleration traces. All values are regenerated from the reference simulation code, not digitized from published figures, and they serve as a reusable ground truth for learning.
2) A systematic benchmark of eighteen supervised regressors from six model families that predict the capacity of the spectrum-allocation search from thirteen physical channel features. The evaluation uses leakage-safe nested cross-validation and re-

ports how often each model exceeds the physical capacity bound.

3) A D3QN channel-selection policy that operates on the same individual channels as MaxActive. It is contrasted with the regressors as an example of learning the allocation *decision* rather than its *value*, and its per-context training requirement is stated explicitly.
4) A convergence and complexity analysis that combines asymptotic cost with measured execution time and memory footprint. The analysis separates model inference from the feature extraction that precedes it, so the reported speed-ups compare like with like.

The remainder of this paper is organized as follows. Section II presents the THz UAV channel model, formulates the spectrum allocation problem, and reviews the two existing solutions, MaxActive and the D3QN policy. Section III describes the alternative ML solutions and how they use the physical channel state. Section IV gives the simulation parameters, the data generation, and the results. Section V analyzes convergence and complexity, and Section VI concludes the paper.

II. SYSTEM MODEL AND BACKGROUND

A. THz UAV Channel Model

We consider a link between a transmitting (Tx) and a receiving (Rx) UAV, as in [9]. At time t the two UAVs are at the three-dimensional positions $\mathbf{p}_{\mathrm{Tx}}(t)$ and $\mathbf{p}_{\mathrm{Rx}}(t)$, separated by the distance $d = \|\mathbf{p}_{\mathrm{Rx}}(t) - \mathbf{p}_{\mathrm{Tx}}(t)\|$. The time index is dropped below when no confusion arises. Either UAV may move. In the scenarios studied here, the Tx hovers and the Rx moves, except for the real-trace scenario, where both move.

Frequency channels. The THz band $[f_{\min}, f_{\max}]$ is divided into a set $\mathcal{F}$ of narrowband channels. Channel $i \in \mathcal{F}$ has center frequency f_i and width Δf_i, which equals the spectral resolution of the atmospheric data. A spectrum allocation activates a subset $\Omega \subseteq \mathcal{F}$ and assigns a transmit power $P_i \geq 0$ to each active channel.

Path loss. The path loss of channel i is the product of a spreading loss and an absorption loss [6]. The spreading (free-space) loss is the Friis term, which in decibels, with d in meters and f_i in megahertz, reads

$$L_{\mathrm{spr},i}(d) = 20\log_{10} d + 20\log_{10} f_i - 27.55. \tag{1}$$

The absorption loss follows from the Beer–Lambert law. The fraction of power that passes through a path of length d is the transmittance

$$\tau_i(d) = e^{-k_i d}, \tag{2}$$

where k_i is the molecular absorption coefficient of the atmosphere at frequency f_i. We obtain k_i from transmittance values computed by the line-by-line radiative transfer model (LBLRTM) [20] for the UAV altitude, as in [9]. The absorption loss is $L_{\mathrm{abs},i}(d) = 10\log_{10}\big(1/\tau_i(d)\big) = 10\,k_i d\log_{10} e$ dB, and the total path loss in linear scale is

$$A_i(d) = 10^{\,[L_{\mathrm{spr},i}(d) + L_{\mathrm{abs},i}(d)]/10}. \tag{3}$$

Because k_i varies by orders of magnitude between neighboring channels, A_i is highly frequency selective. Moreover, because $L_{\mathrm{abs},i}$ grows linearly with d, the transmission windows narrow as the UAVs move apart.

Molecular absorption noise. The molecules that absorb THz energy re-emit it as noise. The emissivity of the path is $1 - \tau_i(d)$, and the noise power in channel i is [6]

$$P_{N,i}(d) = k_B\, T_0\, \big(1 - \tau_i(d)\big)\, \Delta f_i, \tag{4}$$

where k_B is the Boltzmann constant and T_0 is the reference temperature. The noise is therefore strongest exactly where the absorption is strongest.

Antenna gain and beam misalignment. Both UAVs use the three-dimensional spherical-lobe (sectored) antenna model of [9], with the same beamwidth ψ (in radians) in elevation and azimuth. The antenna of node $a \in \{\mathrm{Tx}, \mathrm{Rx}\}$ has the gain

$$G_a = \begin{cases} 4\pi/\psi^2, & |\Delta\theta_a| \leq \psi/2 \text{ and } |\Delta\phi_a| \leq \psi/2, \\ \rho, & \text{otherwise}, \end{cases} \tag{5}$$

where $\Delta\theta_a$ and $\Delta\phi_a$ are the azimuth and elevation (zenith) angles between the antenna boresight and the instantaneous line of sight to the other UAV, and $\rho \ll 1$ is the side-lobe level. When the boresight points to the zenith or the nadir, the azimuth is undefined and only the elevation condition is applied. The boresights are fixed by the initial geometry. As the UAVs move, the line of sight rotates away from the boresights, and once it leaves the main lobe the gain drops from $4\pi/\psi^2$ to ρ. This is *beam misalignment*. Widening the beam (*beamwidth adjustment*) keeps the link in the main lobe, at the price of a lower gain $4\pi/\psi^2$. The total link gain is $G = G_{\mathrm{Tx}} G_{\mathrm{Rx}}$.

Per-channel SNR and capacity. With power P_i on channel i, the signal-to-noise ratio (SNR) of the channel is

$$\gamma_i = \frac{P_i\, G}{A_i(d)\, P_{N,i}(d)}. \tag{6}$$

The SNR thus combines all the physical effects above: distance and frequency through A_i, the atmosphere through A_i and $P_{N,i}$, and geometry and beam alignment through G. Treating each channel as an additive white Gaussian noise channel, the link capacity of an allocation $(\Omega, \{P_i\})$ is the Shannon sum

$$C(\Omega, \{P_i\}) = \sum_{i \in \Omega} \Delta f_i\, \log_2(1 + \gamma_i)\,. \tag{7}$$

Power allocation. Two power-allocation policies are considered. Under EP, the total transmit power P_T is split evenly across the active channels,

$$P_i = P_T/|\Omega|, \qquad i \in \Omega. \tag{8}$$

Under WF, which maximizes (7) for a fixed set Ω [21],

$$P_i = \max\big(0,\ \mu - A_i P_{N,i}/G\big), \qquad i \in \Omega, \tag{9}$$

where the water level μ is chosen so that $\sum_{i\in\Omega} P_i = P_T$. WF gives more power to channels with low attenuation

and noise and switches off channels whose effective noise $A_i P_{N,i}/G$ is above the water level. EP is simpler to implement because it needs no per-channel power control.

B. Problem Formulation

Following [9], the spectrum allocation problem at time t is to choose the active channel set, the power allocation, and the beamwidth that maximize the link capacity:

$$\max_{\Omega\subseteq\mathcal{F},\,\{P_i\},\,\psi} \quad C(\Omega,\{P_i\}) \tag{10a}$$

$$\text{s.t.} \quad \textstyle\sum_{i\in\Omega} P_i \le P_T, \tag{10b}$$

$$P_i \ge 0,\ i\in\Omega; \qquad P_i = 0,\ i\notin\Omega, \tag{10c}$$

$$\psi_0 \le \psi \le \psi_{\max}, \tag{10d}$$

where ψ_0 is the nominal beamwidth and $\psi_{\max}$ the widest admissible one. The capacity depends on ψ through G in (6). The problem is combinatorial in Ω, which has $2^{|\mathcal{F}|}$ candidate subsets, and $|\mathcal{F}|$ is in the thousands in the THz band. It is also coupled, because the best channels depend on the powers and vice versa. An exhaustive search is therefore infeasible [9]. Problem (10) is solved for every geometry, so its solution must be recomputed as the UAVs move.

Reference [9] decomposes (10) into two stages. *(i) Beamwidth adjustment:* ψ is chosen first so that the link stays in the main lobe with the largest gain. *(ii) Channel selection and power allocation:* for the resulting gain, a channel-selection rule and the power allocation (8) or (9) are iterated until the channel set they produce no longer changes. The spectrum-allocation *decision* studied in this paper is the output of stage (ii): the set Ω and the capacity C that it delivers.

C. Existing Solution: MaxActive

MaxActive [9] solves stage (ii) by a threshold search on the SNR. The channels are ranked by SNR, and an SNR threshold is lowered from the largest SNR in decrements Δs. At each of the Q threshold levels, the candidate set Ω_j contains the channels whose SNR exceeds the threshold. The power allocation is applied to Ω_j, and its capacity (7) is evaluated. MaxActive returns the candidate set with the largest capacity. Because the candidates are SNR-ordered prefixes $\Omega_1 \subset \Omega_2 \subset \cdots$, the search examines at most $|\mathcal{F}|$ sets rather than $2^{|\mathcal{F}|}$. Each evaluation sums up to $|\mathcal{F}|$ terms, which gives a per-update cost of $O(Q\,|\mathcal{F}|)$; [9] gives a detailed worst-case operation count. In the joint scheme, MaxActive and the power allocation run in alternation until their channel sets agree. Reference [9] showed that MaxActive with EP closely approaches the WF capacity and is the most practical of the schemes it considered.

In what sense MaxActive is optimal. The search returns the best member of the family of SNR-ordered nested subsets, for the given channel state and power allocation. This optimality has two limits. First, it holds only within the adopted threshold-search family. Subsets that are not SNR-ordered prefixes are never examined, so global optimality over all $2^{|\mathcal{F}|}$ subsets is not claimed in general. Under EP with (near-)equal channel widths, every selected channel receives the same power, and the capacity-maximizing subset of any given size then consists of the highest-SNR channels. In that case, restricting the search to SNR-ordered prefixes loses nothing, provided the search examines every prefix length. Second, selection and power allocation are optimized alternately rather than jointly, so MaxActive is optimal *conditioned* on the power allocation supplied to it, not for the joint selection-and-power problem (10). MaxActive serves in this restricted sense, within the LBLRTM channel model and the spherical-lobe antenna model, as the learning target and the reference throughout this paper. Its cost is the motivation for learning: the threshold search re-evaluates the capacity over every channel of a wide band at every geometry update, on a processor that also flies the aircraft.

D. Existing Learning-Based Solution: D3QN Channel Selection

Elçik *et al.* [17] replaced the threshold search by a learned policy. In deep Q-learning [22], a neural network with weights $\boldsymbol{\theta}$ approximates the action-value function $Q(\mathbf{s},a)$, the expected return of action a in state $\mathbf{s}$, and the agent acts greedily, $a^\star = \arg\max_a Q(\mathbf{s},a;\boldsymbol{\theta})$. Two refinements give the D3QN. The double-DQN target [23]

$$y = r + \beta\, Q\big(\mathbf{s}', \arg\max_{a'} Q(\mathbf{s}',a';\boldsymbol{\theta});\boldsymbol{\theta}^-\big) \tag{11}$$

uses the online weights $\boldsymbol{\theta}$ to choose the next action and a separate target network $\boldsymbol{\theta}^-$ to evaluate it, which reduces overestimation. Here r is the reward, $\mathbf{s}'$ the next state, and β the discount factor. The dueling architecture [24] splits the network into a state-value stream $V(\mathbf{s})$ and an advantage stream $A(\mathbf{s},a)$, and combines them as

$$Q(\mathbf{s},a) = V(\mathbf{s}) + \Big(A(\mathbf{s},a) - \frac{1}{|\mathcal{A}|}\sum_{a'\in\mathcal{A}} A(\mathbf{s},a')\Big), \tag{12}$$

where $\mathcal{A}$ is the action set. Subtracting the mean advantage makes the decomposition identifiable.

In [17], the state collects the path loss and the noise power of every channel. An action a selects a number of channels, and that many highest-SNR channels are activated with EP. The reward is a logarithmic transform of the achieved capacity (given in Section III-D). Because the selected channels do not change the next channel state, the problem was cast as a contextual bandit [25]. One network is pre-trained for each distance and total-gain pair. Over 0.5–1 THz, the learned policy matched the MaxActive capacity with about 10^4 times fewer operations. The present paper re-implements this policy over the full 0.75–4.4 THz band of [9] and compares it with supervised alternatives (Section III).

III. ALTERNATIVE MACHINE LEARNING SOLUTIONS

A. Two Ways to Learn the Spectrum Allocation

All learned models in this work use the same physical channel state as MaxActive: for each operating context, defined by a UAV geometry and the resulting gain G, beamwidth ψ, and distance d, the per-channel path losses $\{A_i\}$ and noise powers $\{P_{N,i}\}$ of Section II-A. Two learning targets are possible. A *capacity regressor* learns the *value* of the spectrum-allocation decision: it maps a compact feature vector $\mathbf{x}$ of the channel state to an estimate $\hat{C}$ of the capacity that MaxActive would achieve, in the spirit of [13]. A *selection policy* learns the *decision* itself: it maps the channel state to a set of active channels, whose capacity is then evaluated with the physical model (6)–(7). The distinction matters in practice. A regressor returns a number, not a channel set, so it can predict a capacity that no feasible allocation reaches. A policy returns an allocation that can be transmitted, and the capacity reported for it is the capacity that allocation actually delivers.

Because MaxActive returns the best capacity within its search family, it upper-bounds any model trained to imitate it. When regressor outputs are *plotted* against the reference, we therefore display

$$\hat{C}_{\text{clip}} = \min\big(\hat{C},\, C_{\text{MaxActive}}\big). \tag{13}$$

The clip (13) requires $C_{\text{MaxActive}}$, which is exactly what a deployed regressor does not have; if it were available online, no surrogate would be needed. *The clip is therefore used only for offline visualization.* It is not part of the online prediction path. At deployment, the regressor emits $\hat{C}$ and the policy emits a channel set, with no access to the reference. Every error reported in Section IV-B is computed from raw, unclipped predictions, and the ceiling violations that (13) would hide are reported explicitly.

B. Physical Feature Vector and Learning Target

The regressors do not see the $|\mathcal{F}|$ individual channels. Instead, each operating context is summarized by a 13-dimensional feature vector built from quantities of the channel model. The band is divided into $K = 20$ equal-width sub-bands. For each sub-band k, we compute the mean total path loss $\bar{L}_k$ (in decibels), the mean absorption-noise power $\bar{P}_{N,k}$ from (4), and a received-power indicator $\Pi_k = \log_{10}\big(P_T\, G\,/\,10^{\bar{L}_k/10}\big)$. Π_k is the numerator of the SNR (6) under full power, expressed in $\log_{10}$ watts. The features are:

- *Geometry and antenna:* $\log_{10}(d+1)$ and its square; the normalized total gain $G_{\text{dB}}/60$ and its square; and the normalized beamwidth $\psi/360°$. These features capture the distance dependence of (1) and the alignment state of (5).
- *Channel quality across the band:* the mean and the standard deviation over sub-bands of Π_k and of $\log_{10}\bar{P}_{N,k}$; the normalized mean path loss $\overline{L}/200$; and the fraction of sub-bands that contain channels. The means describe how strong the link is. The standard deviations describe how frequency selective it is, which governs how many channels are worth activating.
- *Context labels:* a mobility-scenario index and a power-allocation flag. The allocation flag is constant within each training set, because WF and EP are learned as separate targets.

The learning target is the base-10 logarithm of the MaxActive capacity in gigabits per second, $y = \log_{10}(C_{\text{MaxActive}}/1\text{ Gbps})$, computed separately for WF and EP. The logarithm is essential. Across the scenarios, the capacity spans about nine orders of magnitude (Section IV-B), and on that scale the capacity curve is smooth. For the models that are sensitive to feature scale, each feature is standardized to zero mean and unit variance using statistics of the training data only.

C. Supervised Capacity Regressors

Each regressor learns a mapping $f(\mathbf{x}) \approx y$ from labeled examples $\{(\mathbf{x}_n, y_n)\}$ generated by the reference model, using its own standard training objective. To avoid committing to one hypothesis class in advance, we evaluate eighteen regressors in six families [26], [27]. The families, their inductive biases, and their computational character are as follows.

Linear and regularized models (ordinary least squares, LinearReg; Ridge; Lasso; ElasticNet; BayesianRidge) assume that y is affine in $\mathbf{x}$. In the log domain, the capacity is approximately affine in log-distance and gain, so these models are natural baselines. The ℓ_2, ℓ_1, elastic-net, and evidence-based variants separate the bias of the affine hypothesis from the bias of the penalty. Training costs $O(nd^2)$ for n samples and d features, and a prediction is a d-term dot product. *Robust linear regression* (Huber) uses a loss that is quadratic for small residuals and linear for large ones, which tests sensitivity to the extreme dynamic range.

Kernel and probabilistic models (support vector regression with a radial basis function kernel, SVR-RBF; kernel ridge regression, KernelRidge; and a Gaussian process, GaussProc) model smooth nonlinear functions through a kernel that measures the similarity of two operating contexts. The Gaussian process [28] uses a squared-exponential kernel plus a white-noise term and returns the posterior mean. Its kernel hyperparameters are set by maximizing the marginal likelihood. These models suit a smooth capacity surface, but their training cost grows as $O(n^2)$ to $O(n^3)$ with the number of samples.

Instance-based learning (k-nearest neighbors, k-NN) predicts the capacity as an average over the most similar training contexts. It interpolates locally and needs no training, but every prediction scans the stored data.

Neural networks (a multilayer perceptron, MLP) are flexible function approximators trained by gradient descent on the squared error.

Tree ensembles (RandomForest, ExtraTrees, Bagging, AdaBoost, gradient boosting, GradBoost, and XGBoost [27]), together with a single DecisionTree as an unregularized contrast, partition the feature space into regions with a constant prediction in each. They are strong learners for tabular data, but a piecewise-constant function cannot extrapolate a curve that spans many decades.

A histogram-based gradient-boosting regressor was also implemented and then discarded. It is designed for large binned datasets and gives a negative cross-validated R^2 on 33 samples. Because models within a family behave alike here while the families differ by orders of magnitude in error (Section IV-B), adding more members of an existing family is unlikely to change the ranking, whereas omitting a family could.

Interpretation and limitation. The regressor output $\hat{C}$ estimates the capacity that the spectrum-allocation search would deliver at the current geometry, which can be used, for example, in link-budget and rate decisions. A regressor does *not* return the channel set Ω. Where the set itself is needed, the policy of Section III-D must be used. Because a regressor is not tied to a feasible allocation, nothing prevents $\hat{C}$ from exceeding the physical bound; how often this happens is a first-class evaluation criterion.

D. D3QN Channel-Selection Policy

The decision branch re-implements the D3QN of [17] at the same individual-channel granularity as MaxActive.

State. The channels are ranked by $1/(A_i P_{N,i})$, which is the SNR order (6) at a common power level. The state stacks the log path loss and the log noise power of the N best-ranked channels:

$$\mathbf{s} = \big[\log_{10} A_{(1)}, \ldots, \log_{10} A_{(N)}, \log_{10} P_{N,(1)}, \ldots, \log_{10} P_{N,(N)}\big], \tag{14}$$

where the subscript (m) denotes the mth-ranked channel.

Action. An action $a \in \mathcal{A}$ selects a number of channels $N_{\text{sel}}(a)$. The N_{sel} highest-ranked channels are activated with EP (8), so the action space contains only SNR-ordered prefixes, the same family that MaxActive searches. The resulting allocation is evaluated with the physical model (6)–(7), which gives the true capacity C of the decision.

Reward. Following [17], the reward is

$$r = \ln\big((C_{\max} - C_{\min}) + 1\big) - \ln\big((C_{\max} - C) + 1\big), \tag{15}$$

where all capacities are in bit/s. In the implementation, as in [17], $C_{\max}$ is the MaxActive-EP capacity of the current context, obtained by searching the same SNR-ordered prefixes, and $C_{\min} = 0$. The reward is therefore $r = \ln(C_{\max} + 1) - \ln(C_{\max} - C + 1)$. It is zero for $C = 0$ and reaches its maximum $\ln(C_{\max} + 1)$ when the policy matches the reference. The logarithms compress the very large capacity range of a THz link. Because $C_{\max}$ enters the reward, *training a context requires the reference capacity of that context*; only the trained policy, not its training, is independent of the search.

Network and training. The action values are produced by the dueling decomposition (12) from a small fully connected trunk. Training alternates ϵ-greedy exploration, which evaluates the capacity of each explored action with the physical model, with gradient steps that regress $Q(\mathbf{s}, a)$ onto the observed reward under a smooth-ℓ_1 loss, using minibatches drawn from a replay memory. After training, the returned decision is

$$a_{\text{out}} = \arg \max_{a \in \{a^\star\} \cup \mathcal{A}_{\text{exp}}} C(a), \qquad a^\star = \arg\max_a Q(\mathbf{s}, a), \tag{16}$$

where $\mathcal{A}_{\text{exp}}$ is the set of actions explored during training and $C(a)$ their evaluated capacities. That is, the greedy action of the trained network is kept unless an action already evaluated during training delivered a higher capacity. This fallback is part of the implementation of the policy and of every D3QN result reported here.

This is a contextual bandit, not sequential reinforcement learning. For a given geometry, the channel state $\mathbf{s}$ is fixed. The agent takes one action, receives one reward, and the episode ends. The selection made now does not change the channel state at the next geometry, which is set by the trajectory and the physics. With no state transition to influence, the horizon is one and the discount β in (11) is zero. The double-DQN target therefore reduces to the immediate reward, $y = r$, and $Q(\mathbf{s}, a)$ is an estimate of the reward of action a. The model is a contextual $|\mathcal{A}|$-armed bandit [25]. The mechanisms that make deep RL fragile, namely bootstrapping bias, moving targets, and delayed rewards, are absent. Because the target is the immediate reward, the separate target network of (11) is not needed and is not used. We keep the dueling architecture and the D3QN name for fidelity to [17], [24], but claim no sequential decision-making.

Interpretation. The policy outputs a channel set that can be transmitted. Its capacity is that of a feasible SNR-ordered prefix with equal power, so it cannot exceed the best such prefix, which is the set MaxActive-EP searches for. Agreement with MaxActive should be read with three qualifications. First, the policy can at best match the reference; it cannot improve on it. Second, its training uses the reference capacity $C_{\max}$ in the reward. Third, by (16), the reported decision is the best of the greedy action and the actions evaluated during training, so the agreement reflects the learned network together with the exploration performed while training it. The implementation does not record which of the two produced the returned action.

Operation at an unseen geometry, and its cost. As in [17], the policy is trained per operating context: one lightweight network for each (distance, gain) pair. This is a real restriction. A previously unseen geometry is not covered by an already-trained network, and obtaining a policy for it requires its own training run, which itself needs the reference capacity of that geometry. That run is short in absolute terms (Section V-B), but it is an offline

TABLE I
Simulation Parameters

Parameter	Value
THz band $[f_{\min}, f_{\max}]$	0.75–4.4 THz
Active channels per context $\lvert\mathcal{F}\rvert$	11 699 ($\approx$11 700)
Channel width Δf_i	$\approx$ 0.312 GHz (LBLRTM resolution)
Atmospheric data	LBLRTM, horizontal path, 100 m altitude
LBLRTM ranges	$1, 11, \ldots, 101$ m (Beer–Lambert rescaling)
Reference temperature T_0	296 K
Total transmit power P_T	24 dBm
Antenna model	spherical lobe, symmetric beams
Nominal beamwidth ψ_0 (Tx, Rx)	10°
Main-lobe link gain at ψ_0	$\approx$ 52.3 dB
Side-lobe level ρ (Tx, Rx)	0.1
Beamwidth search	$[\psi_0, 360^\circ]$: 15-point grid, then 2° steps
Power allocation	EP (8) or WF (9)
Selection/allocation loop	alternate until the sets agree (max. 30 iterations)
Tx initial position	$(0, 10, 100)$ m
Rx initial position (scenarios 1, 2, 4)	$(0, 11, 100)$ m ($d = 1$ m)

cost, and it exceeds the cost of a single MaxActive search. The present D3QN branch therefore suits a trajectory envelope that can be enumerated and trained for in advance, and it pays off only when a trained context recurs. It does not suit an arbitrary geometry met in flight. The supervised branch does not share this restriction: one fitted regressor maps the features of any geometry inside the trained envelope to a capacity estimate. Two routes would lift the restriction, and both are left to future work. One is to condition a single network on the geometry descriptors that the regressors already use; the other is to train a bank of policies offline and select among them by nearest context at run time.

IV. PERFORMANCE ANALYSIS

A. Simulation Parameters and Data Generation

Reference simulator. The physical layer is reproduced in Python from the MATLAB simulation code of [9]. The same code supplies the LBLRTM transmittance files, the antenna model, the initial coordinates and velocities, and the trajectory generators. The LBLRTM files are computed for a horizontal path at 100 m altitude and resolve the band at the ranges $\{1, 11, \ldots, 101\}$ m. For any other range, the transmittance of the nearest file is rescaled to the exact distance by the Beer–Lambert law (2). All capacities used in this paper are regenerated by this code. No value is read off a published figure; where the regenerated value differs from a figure of [9], the regenerated value is kept. The transmit power of 24 dBm is taken from the reference simulation code. The main parameters are listed in Table I.

Channel selection and beamwidth. The MaxActive threshold sweep is implemented as an equivalent sweep over the number of SNR-ranked channels that are retained: a coarse grid is followed by a unit-step refinement around the best value. The power allocation and MaxActive are alternated until the active set is stable. In the scenarios with beamwidth adjustment, the common Tx/Rx beamwidth is searched over $[\psi_0, 360^\circ]$ before each selection, maximizing the EP capacity over the full band.

Mobility scenarios. Four patterns are generated, the last three misaligned. Each pattern is evaluated with WF and EP, and the misaligned ones with and without beamwidth adjustment. The antenna boresights are fixed by the first position of each trajectory.

1) *Aligned:* the Rx recedes along the y-axis at 10 m/s for 11 steps, giving $d = 1, 11, \ldots, 101$ m with the beams aligned throughout.
2) *Misaligned, diagonal mobility:* the Rx recedes with velocity $(5\sqrt{2}, 5\sqrt{2}, 0)$ m/s, so the fixed 10° beams progressively misalign; 11 steps.
3) *Misaligned, mobility over elevation:* the Rx moves on a hemisphere of radius 20 m around the Tx, with the elevation (zenith) angle ϕ swept from 0° to 180° in 10° steps (19 points). The distance is constant, so the capacity change is caused only by misalignment.
4) *Misaligned, real mobility traces:* the Tx and Rx positions are integrated from measured windy-hover acceleration traces (the drone traces of [8], as supplied with the reference code), with an initial diagonal drift of the Rx. The scenario has 11 points over 0–5 s, and the distance varies between 1.0 and 52.9 m.

Datasets. For the supervised study, one sample is built per scenario–geometry pair. The *main dataset* combines the aligned scenario and the diagonal scenario with and without adjustment, $3 \times 11 = 33$ samples. A separate *mobility dataset* contains the elevation (2×19) and real-trace (2×11) points, 60 samples, and is built identically. Each sample carries the 13 features of Section III-B and the targets y for WF and EP. The sample count is set by the reference model rather than by a sampling choice: the LBLRTM files resolve the straight-line geometry at eleven ranges, and every extra sample costs a full physics evaluation. Three properties make this small-sample regime workable. The target is a deterministic, noiseless function of the features, so n controls only the resolution of the input space, not the variance of the labels. The input space is low dimensional and physically smooth, with capacity varying monotonically in distance and misalignment. Finally, the evaluation protocol below is matched to the regime. Generating more data is embarrassingly parallel and linear in the number of operating points, at roughly 95 ms per sample, so denser range grids, more beamwidths, or more links would raise n from tens to 10^4–10^6 at a cost of CPU-hours. The present models are validated as *interpolators* within the envelope of geometries they were trained on, with no claim of extrapolation beyond it.

Evaluation protocol. Thirty-three samples cannot support a single train/test split for ranking eighteen models, so all regression metrics reported for the main dataset use *nested* cross-validation (CV). On the mobility dataset, the models are tuned by the same inner grid search and then fitted on all 60 samples, so the errors reported there are in-sample. An outer repeated k-fold loop (5 folds $\times$ 3 repeats $=$ 15 folds) estimates generalization. On each outer training partition only, an inner 3-fold grid search tunes the hyperparameters, and the feature standardization sits inside the same pipeline, so no information leaks from the test fold. The metrics are the root-mean-square error (RMSE) and the coefficient of determination R^2 on the log target, and the *median relative error* of the capacity in linear Gbps, which avoids the inflation of the mean absolute percentage error at the very small far-distance capacities. The hyperparameter grids are listed in Table II. The curves plotted in the figures come from each model refit on all samples (the estimator one would deploy), clipped at the ceiling (13) for display, and drawn with monotone piecewise-cubic (PCHIP) interpolation in log space, with markers at the computed points. The metrics come from raw out-of-fold predictions.

D3QN settings. The policy of Section III-D uses the top $N = 500$ channels in the state (a 1000-dimensional input) and 600 actions. The first 50 actions select 1 to 50 channels individually, and the remaining 550 are spaced evenly up to the full channel count. The dueling network maps $1000 \to 16 \to 8$ to a value head and a 600-way advantage head. It is trained for 20 epochs of 80 ϵ-greedy actions (Adam, learning rate 10^{-3}, weight decay 10^{-4}), with one network per (distance, gain) context. One gradient step on a replay minibatch of 64 is taken per epoch, i.e., 20 steps per context, while 1600 actions are evaluated. The returned decision follows (16). The reference for the policy is MaxActive-EP over the same SNR-ranked channels. Since WF and EP give nearly equal capacities (Section IV-B), this comparison also tracks the WF reference.

B. Results

Reference capacity behavior. Table III and Fig. 1 give the MaxActive capacity for the aligned and diagonal scenarios. The ordering aligned $>$ adjusted $>$ non-adjusted holds at every distance, and the scenarios together span about nine decades. In the aligned case the capacity decays smoothly with distance, because both the spreading loss (1) and the absorption loss grow with d while the gain stays at its main-lobe value. Under diagonal mobility without adjustment, the line of sight leaves the 10° main lobe after the first step. The link then operates on the side lobes, where the gain falls from about 52 dB to ρ^2 (-20 dB), and the capacity drops by five to seven orders of magnitude relative to the aligned link. Beamwidth adjustment keeps the link in a wider main lobe and recovers more than three of these decades. WF and EP give nearly the same capacity in every scenario. At

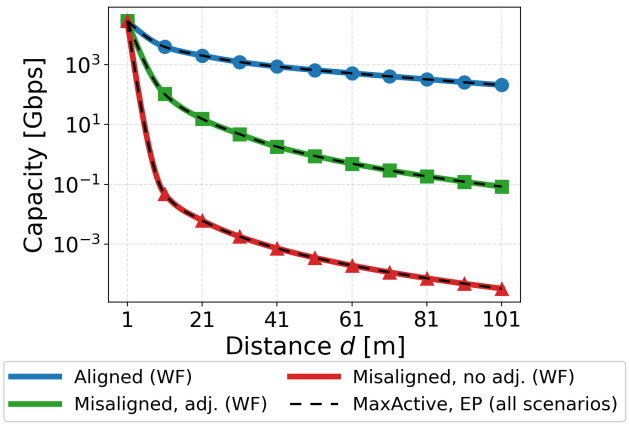


Fig. 1. MaxActive reference capacity versus distance for the aligned link and for diagonal mobility with and without beamwidth adjustment, under WF (thick, solid) and EP (thin, dashed) allocation. The three scenarios occupy disjoint decades of the logarithmic axis, and WF and EP nearly coincide. Curves are PCHIP-smoothed, with markers at the computed distances.

THz frequencies, the channels MaxActive selects already have comparable and high SNR, so water filling adds little over equal power; this confirms the observation of [9] that MaxActive with EP approaches the WF capacity.

Learned models versus the reference. Fig. 2 overlays one representative of each regressor family, together with the D3QN policy, on the MaxActive curves of the aligned and diagonal scenarios. Fig. 3 does the same for the elevation and real-trace scenarios. In the elevation scenario without adjustment, the capacity stays high while the Rx is inside the half-power beam and then drops to a side-lobe plateau once ϕ leaves it. With adjustment, the beam widens and the capacity declines gradually instead. In the real traces, the diagonal drift produces a sharp collapse, and the measured accelerations modulate it with jitter. Because the trajectory begins at the reference initial distance of 1 m, that curve starts from a high capacity. In both figures, the kernel and instance-based representatives follow the reference closely, while the linear representatives lag behind it. The D3QN lies on the MaxActive curve throughout.

Prediction accuracy. Table IV gives the nested cross-validated metrics for the EP target, ranked by RMSE. The WF target was cross-validated in the same way. The two agree to within 0.055 RMSE for every one of the eighteen models, and to within 0.02 for all but the single decision tree, as the near-equality of the WF and EP capacities (Table III) implies. Only the EP column is therefore tabulated. Three findings stand out.

- *The smooth nonparametric and kernel models perform best.* The Gaussian process is best by a wide margin ($R^2 = 0.99$, median relative error $\approx 1.9\%$), followed by ExtraTrees, SVR-RBF, MLP, and k-NN. These models assume the smoothness that the physical capacity surface exhibits, so they interpolate MaxActive closely and generalize across folds.

TABLE II

Learning Settings: Inner-CV Hyperparameter Grids of the Regressors and D3QN Configuration

Family	Model	Inner-CV grid (3-fold) or fitting rule
Linear/regularized	LinearReg	none (ordinary least squares)
	Ridge	$\alpha \in \{0.01, 0.1, 1, 10, 100\}$
	Lasso	$\alpha \in \{10^{-3}, 10^{-2}, 10^{-1}, 1\}$
	ElasticNet	$\alpha \in \{10^{-3}, 10^{-2}, 10^{-1}, 1\}$; ℓ_1 ratio $\in \{0.2, 0.5, 0.8\}$
	BayesianRidge	none (evidence maximization)
Robust	Huber	$\epsilon \in \{1.2, 1.35, 1.5\}$; $\alpha \in \{10^{-4}, 10^{-3}, 10^{-2}\}$
Kernel/probabilistic	SVR-RBF	$C \in \{1, 10, 100\}$; $\gamma \in \{\text{scale}, 0.1, 1\}$; $\epsilon \in \{0.01, 0.1\}$
	KernelRidge (RBF)	$\alpha \in \{0.01, 0.1, 1\}$; $\gamma \in \{0.1, 1, 10\}$
	GaussProc	constant $\times$ squared-exponential $+$ white-noise kernel; marginal-likelihood fit
Instance-based	k-NN	$k \in \{2, 3, 5\}$; uniform or distance weights
Neural	MLP	hidden layers $(64, 32)$ or $(128, 64)$; $\alpha \in \{10^{-4}, 10^{-3}, 10^{-2}\}$; early stopping
Trees	DecisionTree	depth $\in \{2, 3, 5, \text{none}\}$; minimum split $\in \{2, 4, 8\}$
	RandomForest, ExtraTrees	trees $\in \{100, 200\}$; depth $\in \{\text{none}, 5, 10\}$
	AdaBoost	estimators $\in \{50, 100\}$; learning rate $\in \{0.05, 0.1, 0.5\}$
	Bagging	estimators $\in \{50, 100\}$
	GradBoost, XGBoost	trees $\in \{100, 200\}$; depth $\in \{3, 5\}$; learning rate $\in \{0.05, 0.1\}$
D3QN	State	log path loss and log noise of the top $N = 500$ SNR-ranked channels (1000 inputs)
	Actions	600: $N_{\text{sel}} = 1, \ldots, 50$ individually, then 550 values evenly spaced up to $\lvert\mathcal{F}\rvert$
	Network	$1000 \to 16 \to 8$ (ReLU) $\to$ value head (1) and advantage head (600)
	Training	20 epochs $\times$ 80 ϵ-greedy actions, ϵ: $1.0 \to 0.02$ (linear); one replay minibatch (64) per epoch
	Optimizer	Adam, learning rate 10^{-3}, weight decay 10^{-4}; smooth-ℓ_1 loss; gradient-norm clip 1.0
	Reward / output	(15) with $C_{\max}$ = MaxActive-EP capacity, $C_{\min} = 0$; output rule (16)

TABLE III

MaxActive Capacity (Gbps) Versus Distance

	Aligned		Diag. w/ adj.		Diag. no adj.	
d (m)	WF	EP	WF	EP	WF	EP
1	29421	29412	29421	29412	29421	29412
21	2013	2006	15.40	15.34	6.2×10^{-3}	6.2×10^{-3}
61	518.1	516.5	0.491	0.491	1.9×10^{-4}	1.9×10^{-4}
101	208.3	207.5	0.084	0.084	3.3×10^{-5}	3.3×10^{-5}

- *Regularized linear models are stable but biased.* Ridge, Lasso, ElasticNet, and BayesianRidge hold $R^2 \approx 0.97$ with small variance, yet their $\approx 45\%$ median relative error reveals a systematic offset. A model that is affine in the features cannot capture the curvature of a capacity that decays over nine decades.
- *Cross-validation exposes failures that a single split hides.* On a single $75/25$ split, Huber is second only to the Gaussian process (RMSE 0.183), but under CV it falls to $R^2 = 0.93 \pm 0.10$. Ordinary least squares looks acceptable on that split ($R^2 = 0.991$) but is far worse under CV ($R^2 = -7.98 \pm 31$), dominated by unlucky folds. These two, together with the single decision tree and gradient boosting, are the failing methods. On 33 points a single split cannot be trusted; the nested-CV mean $\pm$ std is the basis for the ranking.

Physical-bound violations. The last two columns of Table IV quantify how far the *raw* predictions exceed the reference ceiling. This is what the display clip (13) conceals, and it is the error a deployed regressor would actually commit, since it has no ceiling to clip against.

TABLE IV

Nested Cross-Validated Accuracy and Physical-Ceiling Violations (EP Target, Ranked Best to Worst)

	Nested-CV generalization			Ceiling violations	
Model	RMSE $\pm$ std	R^2	Med. rel.	pts.	max
GaussProc	0.100 ± 0.23	0.991	1.9%	19	2.7%
ExtraTrees	0.237 ± 0.18	0.986	12.1%	1	3.7%
SVR-RBF	0.253 ± 0.22	0.982	7.8%	18	318%
MLP	0.254 ± 0.21	0.980	21.1%	20	226%
BayesianRidge	0.331 ± 0.14	0.976	42.5%	17	322%
k-NN	0.338 ± 0.29	0.959	9.0%	11	111%
XGBoost	0.342 ± 0.16	0.970	36.9%	14	38%
KernelRidge	0.356 ± 0.33	0.962	16.0%	17	223%
Lasso/ENet/Ridge	0.36–0.37	0.97	45–49%	17–19	232–261%
GradBoost	0.412 ± 0.21	0.957	38.4%	16	2993%
RF/Bag./AdaBoost	0.44–0.45	0.96	49–62%	9–11	97–246%
Huber	0.504 ± 0.37	0.930	48.4%	17	1102%
DecisionTree	0.561 ± 0.32	0.931	48.0%	3	3698%
LinearReg	3.281 ± 8.78	-7.98	55.8%	15	405%
D3QN (policy)	—	—	$\sim$0%	0	0%

RMSE and R^2: $\log_{10}$ target; Med. rel.: median relative error in linear Gbps; all from *raw, unclipped* out-of-fold predictions. "pts.": number of the 33 points at which the prediction exceeds the reference; "max": largest such excess, i.e., what (13) conceals. ENet = ElasticNet, RF = RandomForest, Bag. = Bagging. D3QN row: capacity of the selected channel set relative to MaxActive-EP (not a regression metric).

Every regressor in Table IV exceeds the reference at one or more points, so the size of the violation is a first-class selection criterion. The piecewise-constant single tree and plain gradient boosting overshoot by thousands of percent, because they cannot extrapolate the wide dynamic range; a link budget dimensioned on such a prediction would be

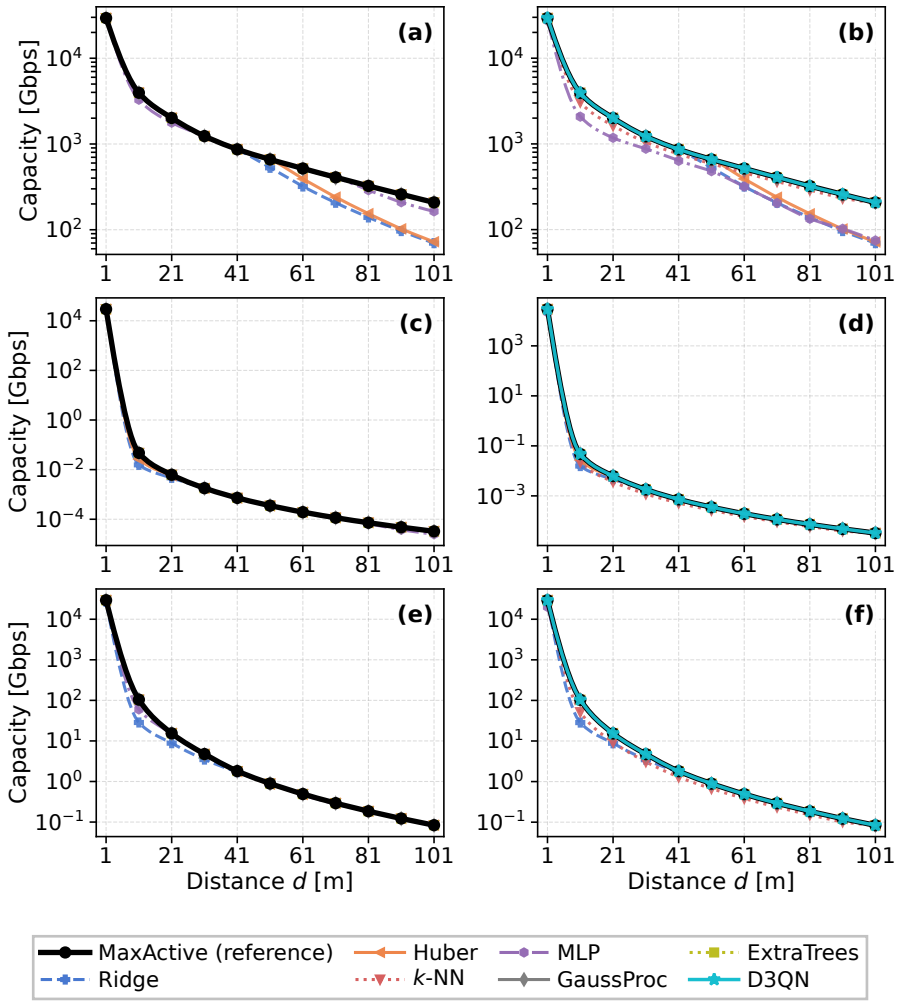


Fig. 2. Learned models versus MaxActive in the aligned and diagonal scenarios. Rows: (a), (b) aligned; (c), (d) misaligned diagonal mobility without beamwidth adjustment; (e), (f) misaligned diagonal mobility with beamwidth adjustment. Columns: WF target in (a), (c), (e) and EP target in (b), (d), (f); the D3QN, which uses EP, appears in the EP column. One regressor per family is shown; all eighteen are compared in Table IV. Regressor curves are models fit on all 33 samples, clipped at the MaxActive ceiling (13) *for display only* (a deployed regressor has no access to this ceiling), and PCHIP-smoothed with markers at the computed points.

badly optimistic. GaussProc and ExtraTrees, by contrast, stay within a few percent. The D3QN capacity did not exceed the MaxActive-EP reference at any of the evaluated geometries, as expected for a policy that selects among the same SNR-ordered prefixes.

Scenario dependence. Table V breaks the out-of-fold median relative error down by scenario for the strongest models. For four of the five models, at least one misaligned scenario is harder than the aligned one; k-NN, for example, rises to 20.1% in the worst-SNR regime (misaligned, no adjustment). The Gaussian process is the exception and stays within 2% in all three scenarios. SVR-RBF and KernelRidge stay within about 7% everywhere, whereas ExtraTrees degrades in both misaligned scenarios. This per-scenario stability, and not merely a low aggregate error, is what qualifies a model as a robust predictor.

Additional mobility patterns. On the elevation and real-trace scenarios (Fig. 3), fitting on all 60 samples, the strongest models reproduce the curves closely: XGBoost, GradBoost, and GaussProc reach median relative errors of 0.07%, 0.7%, and 2.8% over the elevation and real scenarios, with Huber next at 4.4%. This is an *in-sample* fit. It therefore flatters models, such as GradBoost, that fail under cross-validation, which is precisely why Table IV, and not this fit, is the basis for the ranking.

TABLE V
Per-Scenario Median Relative Error (EP Target, Nested-CV Out-of-Fold Predictions)

		Misaligned (diagonal)	
Model	Aligned	No adjustment	With adjustment
GaussProc	2.0%	1.9%	0.8%
SVR-RBF	2.6%	5.3%	6.9%
KernelRidge	4.9%	6.0%	5.3%
k-NN	1.2%	20.1%	0.7%
ExtraTrees	2.7%	12.2%	24.6%

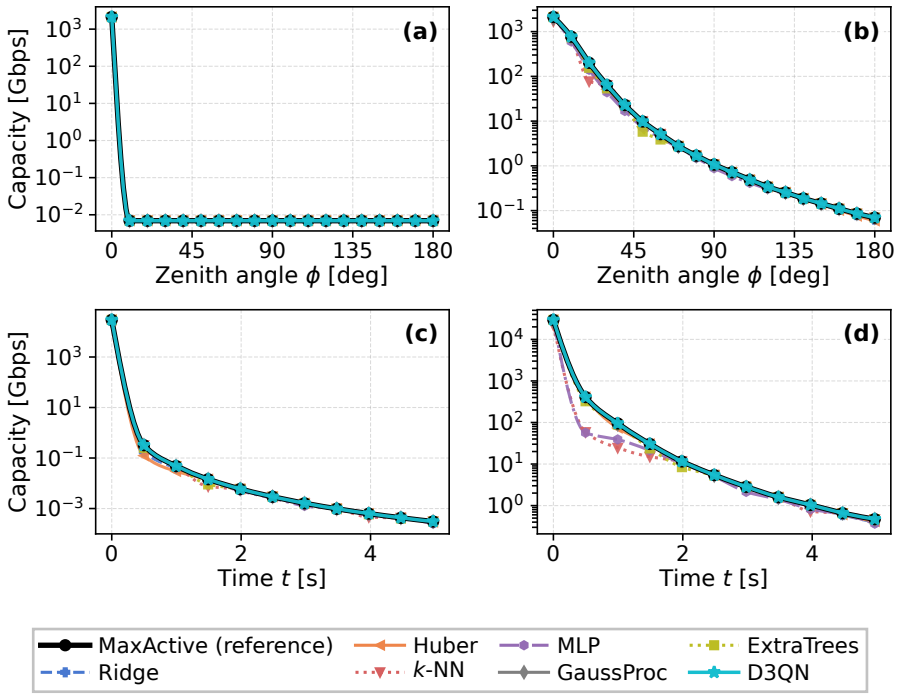


Fig. 3. Learned models versus MaxActive on the two additional mobility patterns (EP target): mobility over elevation (a) without and (b) with beamwidth adjustment, where ϕ is the elevation (zenith) angle of the Rx seen from the Tx; and real windy-hover traces (c) without and (d) with adjustment. Same model representatives as Fig. 2. WF panels are omitted because WF and EP coincide to within the line width. Regressor curves are in-sample fits on the 60 mobility samples, clipped at the ceiling for display only, and PCHIP-smoothed. The D3QN (cyan) lies on the MaxActive curve throughout.

Channel-selection policy. The D3QN, trained as a single-step contextual bandit at every trajectory point (Section III-D), reaches at least 99.7% of the MaxActive-EP capacity on *all seven* scenario variants. The mean capacity ratio is 99.9% for the aligned case and 100.0% for each of the six misaligned cases: diagonal, elevation, and real-trace mobility, each with and without beamwidth adjustment. The worst single operating point over all 93 evaluated geometries (33 in the main study plus 60 mobility points) is 99.7% of the reference, and no point exceeds it. The selected channel *counts* are not always identical to those of MaxActive; in the main study they differ by up to 30 channels (e.g., 497 versus 467 at 81 m in the aligned case). Near its maximum, the capacity is flat in the number of active channels, so neighboring prefixes deliver almost the same capacity. These results show that a policy operating at the same individual-channel granularity as MaxActive-EP returns a transmittable channel set of nearly reference capacity, under elevation and real-trace mobility as well. They must, however, be read with the qualifications of Section III-D. The ratio is bounded by construction, because the policy chooses among the same SNR-ordered prefixes. Training uses the reference capacity in the reward. The reported decision is the better of the greedy action and the best of the 1600 actions evaluated during training (16). The figures above therefore characterize the complete training-plus-selection procedure; how often the greedy network output alone attains them was not recorded.

Discussion and limitations. The contrast between the regressors and the policy is the most transferable finding of this study. A regressor predicts a number and can exceed the physical bound. The policy predicts an allocation, and the capacity it reports is the one its choice actually delivers, so it stays at or below the MaxActive-EP reference (99.7–100%) with no clip anywhere in the pipeline. Where an allocation must be trusted without supervision, learning the decision is the safer target, subject to the per-context training it currently requires. The study also has limitations that bound these claims. The datasets are small (33 and 60 points) and physically smooth, which favors smoothness-assuming models and restricts the validated regime to interpolation within the trained envelope. The learned models imitate a reference that is optimal only within its threshold-search family (Section II-C), so they inherit that scope rather than approximating a globally optimal allocation. The regressors predict the capacity value rather than the channel set, so a system that needs the set must use the policy. The policy, in turn, is trained per context rather than as a single geometry-generalizing agent (Section III-D).

V. CONVERGENCE AND COMPLEXITY

A. Asymptotic Cost

MaxActive has a per-step cost of $O(Q\,|\mathcal{F}|)$ and must be re-run whenever the geometry changes. It needs no

training, but it incurs this search at every time step. The supervised models move the cost offline. The entire nested-CV benchmark of 18 models over both targets completes in about 30 s on a CPU, dominated by the grid searches, and fitting the 18 deployable models on the full dataset takes 0.5 s. Afterwards, a prediction is a single forward, kernel, or neighbor evaluation. Table VI summarizes the asymptotic training and prediction complexity of representative families (n samples, d features, T trees, W network weights, n_{sv} support vectors), together with their measured cost and cross-validated accuracy, and adds the MaxActive and D3QN references for context.

B. Measured Execution Time

Asymptotic orders alone do not establish a practical benefit, so every stage was measured on identical channel states, on a single core of an Intel Core i7-13620H, with the band discretized into $|\mathcal{F}| = 11\,700$ active channels per context. The measured columns of Table VI average the eleven aligned contexts.

The reference costs 77 ms for the threshold sweep alone and 95 ms for a complete link update with power allocation iterated to convergence. On the three geometries where both variants were timed, adding the beamwidth search raises that update from 72 ms to 91 ms, i.e., to $1.3\times$ the fixed-beam cost. Against this, the regressor path costs 0.84 ms to turn the channel state into the 13 features and 21–43 μs to evaluate a linear, kernel, or Gaussian-process model. This is 0.9 ms end-to-end, an $88\times$ reduction in the per-update decision cost. The source of this factor should be stated precisely. The model evaluation alone is 1800–$3700\times$ faster than the search, but feature extraction now dominates the online path, so the end-to-end gain saturates near two orders of magnitude, and *feature extraction, not inference, is the target for further optimization*. The tree ensembles are the exception at the other end. They need 3.0–12 ms per prediction because hundreds of estimators must each be traversed, which cuts their advantage to 6–$20\times$ and makes them a poor per-update choice at this dataset size despite their accuracy. The D3QN is the fastest decision of all. Its state is built from the SNR ranking that MaxActive computes anyway (0.55 ms, shared by both), after which a forward pass and $\arg\max$ take 0.05 ms, three orders of magnitude below the search. It also returns the channel set itself rather than a capacity estimate. Its cost is paid offline: 0.11 s per context, about 4 s for the 33 contexts of the main study. Training one context thus costs more than one threshold sweep (77 ms), and it also needs that context's reference capacity (Section III-D). The per-context policy therefore saves computation only when a trained context is revisited, and the online figure above applies to such contexts.

C. Memory Footprint and Airborne Deployment

For a UAV, memory and predictability matter as much as speed. The deployable models are small: the regularized-linear models occupy 1.2–2.2 KB, SVR-RBF 4.6 KB, the Gaussian process 17.9 KB (its 33×33 kernel and training inputs), the MLP 70 KB, and the D3QN 21 561 parameters, i.e., 84 KB in single precision. These sizes are compatible with the on-chip memory of microcontroller-class flight companions, and all are far smaller than the LBLRTM tables that the reference search itself must hold. Only the tree ensembles are heavy, XGBoost at 522 KB and ExtraTrees at 1.4 MB, which, together with their millisecond-scale inference, makes them a poor fit here despite their accuracy. Three further properties favor deployment. The inference cost is constant and independent of the data: a fixed number of multiply-accumulates with no search, iteration, or early exit. The decision therefore has a hard worst-case latency that a threshold sweep of state-dependent length cannot offer. All training is offline, so the platform performs no gradient computation. Finally, the arithmetic could be compressed further: an 8-bit D3QN would occupy about 21 KB (not evaluated here), and the linear models reduce to a 13-element dot product. On these grounds, the Gaussian process is the better choice where accuracy dominates (18 KB, 43 μs), and the D3QN where the channel set is needed (84 KB, 0.05 ms, subject to the per-context training of Section III-D). The tree ensembles are best reserved for offline or ground-station use. These figures indicate feasibility within the memory and latency budgets of a flight companion. All timings, however, come from a laptop-class CPU, so absolute latencies on an embedded airborne processor will be larger. The *ratio* between search and learned model is expected to carry over approximately, since both run on the same processor, but this has not been measured. A full airborne demonstration remains future work.

D. Convergence

For the supervised models, convergence is that of their standard solvers and is not a limiting factor; the cost lies in the offline grid search. For the D3QN, the single-step formulation of Section III-D removes bootstrapping variance and the credit-assignment horizon, which makes training well conditioned. With the budget of 20 epochs of 80 actions, the returned decision (16) reached 99.7–100% of the reference capacity in all four mobility patterns. Because (16) also considers the actions explored during training, this budget is a measured sufficient setting for the complete procedure, not a convergence result for the greedy network alone. Learning curves of the greedy action were not recorded and are left to future work. The coupled power-allocation and MaxActive loop inside the reference stops when the two channel sets agree and is capped at 30 iterations (Table I); its iteration counts were not recorded.

TABLE VI
Asymptotic and Measured Cost Versus Accuracy of Representative Models

	Asymptotic cost		Measured (single core, $n = 33$)			Nested-CV accuracy (EP)	
Model (family)	Training	Prediction	Train [ms]	Predict [ms]	Memory [KB]	R^2	Med. rel. err.
Ridge / Lasso (linear)	$O(nd^2)$	$O(d)$	0.7	0.023	1.2	0.97	45%
k-NN (instance-based)	$O(1)$ (store)	$O(nd)$	0.5	0.211	7.6	0.96	9%
DecisionTree (tree)	$O(nd \log n)$	$O(\text{depth})$	0.4	0.029	5.3	0.93	48%
XGBoost / ExtraTrees (ensemble)	$O(T\, nd \log n)$	$O(T\,\text{depth})$	76–103	0.32–11.7	522–1362	0.97–0.99	12–37%
MLP (neural)	$O(\text{it} \cdot nW)$	$O(W)$	49	0.031	70	0.98	21%
SVR-RBF (kernel)	$O(n^2 - n^3)$	$O(n_{sv} d)$	0.8	0.038	4.6	0.98	8%
GaussProc (kernel/prob.)	$O(n^3)$, mem $O(n^2)$	$O(n)$ mean	14	**0.043**	**17.9**	**0.99**	**1.9%**
MaxActive (reference)	— (online search)	$O(Q\|\mathcal{F}\|)$ / step	—	**76.8**	—	reference	—
D3QN (policy)	$O(E \cdot A)$ / context	$O(\text{forward pass})$	115/ctx.	0.05	84	$\approx$ ref.	$\sim$0%

Prediction times are per operating point and exclude the 0.84 ms feature extraction shared by all regressors (Section V-B); the MaxActive row is the threshold sweep alone, on the same channel states. Memory is the serialized model size; for the D3QN it is 21 561 parameters in single precision. For the D3QN, E is the number of training epochs and A the number of actions evaluated per epoch.

E. Accuracy Versus Complexity

A clear accuracy-versus-complexity trade-off emerges, and it is the central practical conclusion of this study. The most accurate model, the Gaussian process, has the highest asymptotic training cost ($O(n^3)$ time, $O(n^2)$ memory, and a prediction that requires the full training kernel). The next best, SVR-RBF and kernel ridge, are likewise kernel based at $O(n^2 - n^3)$, and the cheapest models ($O(nd^2)$ training, $O(d)$ prediction) are systematically biased ($\approx 45\%$ median error). The accuracy ranking of Table IV therefore roughly tracks the asymptotic cost ranking of Table VI: the last few percent of accuracy are bought at a super-linear price in the number of samples.

Two regimes invert the recommendation, however. At the present scale ($n \leq 60$), the asymptotic penalty is invisible in wall-clock terms. The Gaussian process trains in 14 ms and predicts in 43 μs, which is cheaper at inference than k-NN or any ensemble. The most accurate model can therefore be chosen without a practical penalty, and the Gaussian process is the model we recommend at this scale. What costs time here is the number of estimators rather than the asymptotic class, which is why the tree ensembles are the slowest predictors in the table. As n grows toward the 10^4–10^6 samples of Section IV-A, the cubic training cost and quadratic memory of the kernel methods become prohibitive, while the ensembles amortize their per-tree overhead, and the balance shifts to XGBoost and ExtraTrees. A reasonable guideline is therefore to prefer kernel methods while the dataset is small and tree ensembles once it is not.

VI. CONCLUSIONS

This correspondence studied spectrum allocation for THz UAV links, where molecular absorption, absorption noise, and mobility-induced beam misalignment make the choice of active channels and their powers decisive for capacity. Taking the MaxActive threshold search as the reference solution, we investigated two machine-learning alternatives that work from the same physical channel state: supervised regressors that predict the capacity the search achieves, and a D3QN contextual-bandit policy that learns the channel selection itself.

Three findings carry beyond this system. First, the inductive bias that matters is smoothness. The capacity surface is smooth and wide-ranging, so kernel and Gaussian-process models track the reference closely, whereas piecewise-constant learners cannot extrapolate it and overshoot the physical bound by orders of magnitude. Second, on datasets this small, the evaluation protocol decides the ranking. A single split reorders the models and hides overfitting, so nested cross-validation is a requirement rather than a refinement. Third, learning the *decision* is safer than learning its *value*. A policy that selects channels reports only capacities that a transmittable allocation actually delivers, whereas a value regressor can predict a rate the link cannot support.

In terms of cost, once trained, both learned branches lower the online decision cost by about two orders of magnitude or more relative to the search, and they fit in the memory of a small flight computer. For the regressors, feature extraction rather than inference now limits the speed-up. At the present scale, the Gaussian process is the preferred capacity predictor, and the D3QN is preferred where the channel set is needed. Only as the dataset grows by orders of magnitude does the super-linear cost of kernel methods favor tree ensembles. Three caveats bound these conclusions. First, the D3QN branch is trained per operating context with the reference capacity in its reward, and its reported decision also draws on the actions explored in training; it therefore covers a pre-enumerated trajectory envelope rather than an arbitrary unseen geometry, and it pays off only when trained contexts recur. Second, the regressors are validated as interpolators within the trained envelope. Third, all timings come from a laptop-class CPU. Future work will replace the per-context bandits with a single geometry-generalizing policy, evaluate the greedy policy without the training-time fallback, predict the channel set rather than only the capacity, validate on embedded hardware, and

extend the dataset into the regime where the complexity trade-off binds.

KORHAN ERDOĞDU
Sabancı University, Istanbul, Türkiye

ÖZGÜR GÜRBÜZ
Sabancı University, Istanbul, Türkiye

AKHTAR SAEED
Kadir Has University, Istanbul, Türkiye

MIKAIL ERDEM
Sabancı University, Istanbul, Türkiye

REFERENCES

[1] I. F. Akyildiz, C. Han, Z. Hu, S. Nie, and J. M. Jornet
Terahertz band communication: An old problem revisited and research directions for the next decade
IEEE Trans. Commun., vol. 70, no. 6, pp. 4250–4285, Jun. 2022, doi: 10.1109/TCOMM.2022.3171800.

[2] C. Chaccour, M. N. Soorki, W. Saad, M. Bennis, P. Popovski, and M. Debbah
Seven defining features of terahertz (THz) wireless systems: A fellowship of communication and sensing
IEEE Commun. Surveys Tuts., vol. 24, no. 2, pp. 967–993, 2022, doi: 10.1109/COMST.2022.3143454.

[3] M. M. Azari, S. Solanki, S. Chatzinotas, and M. Bennis
THz-empowered UAVs in 6G: Opportunities, challenges, and trade-offs
IEEE Commun. Mag., vol. 60, no. 5, pp. 24–30, May 2022, doi: 10.1109/MCOM.001.2100889.

[4] A. Saeed, O. Gurbuz, and M. A. Akkas
Terahertz communications at various atmospheric altitudes
Phys. Commun., vol. 41, 2020, art. no. 101113, doi: 10.1016/j.phycom.2020.101113.

[5] A. Saeed, O. Gurbuz, A. O. Bicen, and M. A. Akkas
Variable-bandwidth model and capacity analysis for aerial communications in the terahertz band
IEEE J. Sel. Areas Commun., vol. 39, no. 6, pp. 1768–1784, Jun. 2021, doi: 10.1109/JSAC.2021.3071831.

[6] J. M. Jornet and I. F. Akyildiz
Channel modeling and capacity analysis for electromagnetic wireless nanonetworks in the terahertz band
IEEE Trans. Wireless Commun., vol. 10, no. 10, pp. 3211–3221, Oct. 2011, doi: 10.1109/TWC.2011.081011.100545.

[7] C. Han, A. O. Bicen, and I. F. Akyildiz
Multi-wideband waveform design for distance-adaptive wireless communications in the terahertz band
IEEE Trans. Signal Process., vol. 64, no. 4, pp. 910–922, Feb. 2016, doi: 10.1109/TSP.2015.2498133.

[8] S. K. Moorthy and Z. Guan
Beam learning in mmWave/THz-band drone networks under in-flight mobility uncertainties
IEEE Trans. Mobile Comput., vol. 21, no. 6, pp. 1945–1957, Jun. 2022, doi: 10.1109/TMC.2020.3034064.

[9] A. Saeed, M. Erdem, A. Saleem, O. Gurbuz, and M. A. Akkas
Joint resource allocation for terahertz band drone communications
IEEE Trans. Veh. Technol., vol. 73, no. 6, pp. 8281–8294, Jun. 2024, doi: 10.1109/TVT.2024.3354067.

[10] A. Shafie, N. Yang, S. A. Alvi, C. Han, S. Durrani, and J. M. Jornet
Spectrum allocation with adaptive sub-band bandwidth for terahertz communication systems
IEEE Trans. Commun., vol. 70, no. 2, pp. 1407–1422, Feb. 2022, doi: 10.1109/TCOMM.2021.3139887.

[11] L. Xu *et al.*
Joint location, bandwidth and power optimization for THz-enabled UAV communications
IEEE Commun. Lett., vol. 25, no. 6, pp. 1984–1988, Jun. 2021, doi: 10.1109/LCOMM.2021.3064067.

[12] S. K. Moorthy and Z. Guan
LeTera: Stochastic beam control through ESN learning in terahertz-band wireless UAV networks
In *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2020, pp. 1039–1044, doi: 10.1109/INFOCOMWKSHPS50562.2020.9162766.

[13] H. Sun, X. Chen, Q. Shi, M. Hong, X. Fu, and N. D. Sidiropoulos
Learning to optimize: Training deep neural networks for interference management
IEEE Trans. Signal Process., vol. 66, no. 20, pp. 5438–5453, 2018, doi: 10.1109/TSP.2018.2866382.

[14] N. C. Luong *et al.*
Applications of deep reinforcement learning in communications and networking: A survey
IEEE Commun. Surveys Tuts., vol. 21, no. 4, pp. 3133–3174, 2019, doi: 10.1109/COMST.2019.2916583.

[15] Y. S. Nasir and D. Guo
Multi-agent deep reinforcement learning for dynamic power allocation in wireless networks
IEEE J. Sel. Areas Commun., vol. 37, no. 10, pp. 2239–2250, Oct. 2019, doi: 10.1109/JSAC.2019.2933973.

[16] A. Shafie, C. Li, N. Yang, X. Zhou, and T. Q. Duong
An unsupervised learning approach for spectrum allocation in terahertz communication systems
In *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2022, pp. 3447–3453, doi: 10.1109/GLOBECOM48099.2022.10001337.

[17] K. B. Elçik, A. Saeed, Ö. Gürbüz, and Ç. Tunç
Spectrum allocation via deep Q-learning for 6G terahertz band drone communications
In *Proc. 33rd Signal Process. Commun. Appl. Conf. (SIU)*, 2025, pp. 1–4, doi: 10.1109/SIU66497.2025.11112483.

[18] Z. Hu and C. Han
Graph neural network aided deep reinforcement learning for resource allocation in dynamic terahertz UAV networks
arXiv:2505.04981, May 2025.

[19] A. Farhad and J.-Y. Pyun
Terahertz meets AI: The state of the art
Sensors, vol. 23, no. 11, 2023, art. no. 5034, doi: 10.3390/s23115034.

[20] S. A. Clough *et al.*
Atmospheric radiative transfer modeling: A summary of the AER codes
J. Quant. Spectrosc. Radiat. Transfer, vol. 91, no. 2, pp. 233–244, 2005, doi: 10.1016/j.jqsrt.2004.05.058.

[21] A. Goldsmith
Wireless Communications. New York, NY, USA: Cambridge Univ. Press, 2005.

[22] V. Mnih *et al.*
Human-level control through deep reinforcement learning
Nature, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

[23] H. van Hasselt, A. Guez, and D. Silver
Deep reinforcement learning with double Q-learning
In *Proc. AAAI Conf. Artif. Intell.*, vol. 30, no. 1, 2016, doi: 10.1609/aaai.v30i1.10295.

[24] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas
Dueling network architectures for deep reinforcement learning
In *Proc. 33rd Int. Conf. Mach. Learn. (ICML), PMLR*, vol. 48, 2016, pp. 1995–2003.

[25] J. Langford and T. Zhang
The epoch-greedy algorithm for multi-armed bandits with side information
In *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 20, 2007.

[26] F. Pedregosa *et al.*

Scikit-learn: Machine learning in Python
J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[27] T. Chen and C. Guestrin
XGBoost: A scalable tree boosting system
In *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[28] C. E. Rasmussen and C. K. I. Williams
Gaussian Processes for Machine Learning. Cambridge, MA, USA: MIT Press, 2006.